

机器学习实验指导手册

感知机模型

一、实验目的

1. 掌握感知机模型的数学原理与学习算法
 2. 理解梯度下降法（整体/随机/批量）的差异及学习率的影响
 3. 实现感知机的对偶形式并分析计算效率
 4. 可视化训练过程中经验风险变化与超平面动态调整
-

二、实验原理

1. 数学模型

原始形式

假设函数：

$$f(x) = \text{sign}(w \cdot x + b)$$

损失函数（误分类点到超平面的距离和）：

$$L(w, b) = - \sum_{x_i \in M} y_i (w \cdot x_i + b)$$

其中， M 为误分类样本集合。

对偶形式

通过引入拉格朗日乘子 α_i ，模型参数可表示为：

$$w = \sum_{i=1}^N \alpha_i y_i x_i, \quad b = \sum_{i=1}^N \alpha_i y_i$$

决策函数变为：

$$f(x) = \text{sign}\left(\sum_{i=1}^N \alpha_i y_i x_i \cdot x + b\right)$$

2. 参数更新方法

- 整体梯度下降（BGD）：每次迭代使用全部样本计算梯度
- 随机梯度下降（SGD）：每次迭代随机选取一个样本更新参数
- 批量梯度下降（Mini-batch GD）：每次使用一个小批量样本计算梯度

3. 学习率设定

- 固定学习率： $\alpha = 0.1$
- 自适应学习率： $\alpha_t = \frac{\alpha_0}{1 + \text{decay_rate} \times t}$ （其中，decay_rate 为事先指定，t 为迭代的第几步）

三、实验步骤

1. 数据准备

使用线性可分数据集（示例代码）：

```
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, random_state=42)

y = np.where(y == 0, -1, 1) # 标签映射为{-1, 1}
```

2. 数据预处理

标准化并划分训练集/测试集：

```
from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

3. 模型实现

原始形式（SGD）：

```

class Perceptron:
    def __init__(self, lr=0.01, max_iter=1000):
        self.lr = lr
        self.max_iter = max_iter
    def fit(self, X, y):
        self.w = np.zeros(X.shape[1])
        self.b = 0
        self.losses = []
        for _ in range(self.max_iter):
            loss = 0
            for xi, yi in zip(X, y):
                if yi * (np.dot(self.w, xi) + self.b) <= 0:
                    self.w += self.lr * yi * xi
                    self.b += self.lr * yi
                    loss += 1
            self.losses.append(loss)
            if loss == 0:
                break
        return self

```

对偶形式:

```

class DualPerceptron:
    def __init__(self, lr=0.01, max_iter=1000):
        self.lr = lr
        self.max_iter = max_iter
    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.alpha = np.zeros(n_samples)
        self.b = 0

```

```

        self.gram = X.dot(X.T) # Gram 矩阵
        for _ in range(self.max_iter):
            for i in range(n_samples):
                if y[i] * (np.sum(self.alpha * y * self.gram[i]) + self.
b) <= 0:

                    self.alpha[i] += self.lr
                    self.b += self.lr * y[i]

            return self

```

4. 可视化训练过程

损失曲线:

```

plt.plot(range(len(perceptron.losses)), perceptron.losses)
plt.xlabel('Epoch')
plt.ylabel('Number of Misclassifications')
plt.title('Training Loss Curve')
plt.show()

```

超平面动态变化:

```

def plot_decision_boundary(model, X, y):
    x1_min, x1_max = X[:,0].min()-1, X[:,0].max()+1
    x2_min, x2_max = X[:,1].min()-1, X[:,1].max()+1
    xx1, xx2 = np.meshgrid(np.linspace(x1_min, x1_max, 100), np.linspace(x2_min, x2_max, 100))
    Z = model.predict(np.c_[xx1.ravel(), xx2.ravel()])
    Z = Z.reshape(xx1.shape)
    plt.contourf(xx1, xx2, Z, alpha=0.3)
    plt.scatter(X[:,0], X[:,1], c=y, edgecolors='k')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.title('Decision Boundary')

```

```
plt.show()
```

四、实验要求

必做任务

1. 实现原始形式（SGD/BGD/Mini-batch GD）与对偶形式的感知机
2. 比较不同学习率（0.001, 0.01, 0.1）对收敛速度的影响
3. 绘制经验损失下降曲线与超平面动态变化图
4. 分析对偶形式在计算效率上的优势

扩展任务

1. 尝试自适应学习率策略
 2. 实现多分类感知机（OVA（one vs all）/ OvO（one vs one）策略）
-

五、实验分析

测试案例

生成线性不可分数据

```
X, y = make_classification(n_samples=100, n_features=2, n_redundant=0,  
flip_y=0.3, random_state=42)
```

```
y = np.where(y == 0, -1, 1)
```

预期结果

- 原始形式在噪声数据下无法完全收敛
- 对偶形式在 Gram 矩阵支持下高效计算

提示：调试时可设置 `max_iter=10` 并打印参数变化观察训练过程。使用断言验证预测结果的一致性。

示例：`assert np.all(perceptron.predict(X_train) == y_train)`

六、思考题

1. 对偶形式中 Gram 矩阵的作用是什么？其空间复杂度如何？

2. 随机梯度下降与批量梯度下降在收敛稳定性上的差异原因?
3. 学习率设置过大或过小会导致什么问题?